Department IV
Data Science / Applied Statistics
Winter Term 2021 / 2022

Term Paper

Reinforcement Learning for Stock Market Investment

Advisors:
M.Sc. Christopher Caratiola
Dr. Florian Ertz
Prof. Dr. Ralf Münnich

submitted on: March 30, 2022

submitted by:
B.Sc. Stavre, Xhorxhia
B.Sc. Maini, Deepanshu
B.Sc. Le, Thao Nguyen

Abstract

Written by: Xhorxhia Stavre

This work explores the potential of Deep Reinforcement Learning methods applied to financial markets and, more specifically, in the portfolio management problem. Two different DRL methods, namely, Proximal Policy Optimization (PPO) and Deep Deterministic Policy Gradient (DDPG) were tested on a simulated stock market environment, adopting the implementations from the FinRL library (Liu et al., 2020). The main goal of the models trained is to maximize the returns by taking the best actions and adjusting the portfolio weights guided by a policy.
The Eurozone ("EZU"), U.S. market (S&P500/"IVV") and Emerging Markets ("EEM") ETFs are included in the study's dataset, from 2011 to 2021.

The results show that in terms of returns and risk-adjusted returns, the DDPG portfolio outperforms the PPO portfolio. However, the difference is relatively small. Moreover, the models seem predict the market trends correctly, since we can clearly see the Covid-19 crisis in March 2020 and the large market gains in Novemeber 2020.

Contents

List of Figures **III**

List of Tables **III**

List of Abbreviations **IV**

List of Notation **V**

Author overview **VI**

1 Introduction **1**
1.1 Problem Statement . . . 2
1.2 Motivation . . . 2
1.3 Report Structure . . . 3

2 Reinforcement Learning Background **3**
2.1 Reinforcement Learning: Key concepts and terminology . . . 5
2.1.1 States and Observations . . . 5
2.1.2 Action Space . . . 5
2.1.3 Policies . . . 6
2.1.4 Trajectory/ Episodes . . . 6
2.1.5 Reward and Return . . . 7
2.1.6 Value Functions . . . 7
2.1.7 Markov Decision Process . . . 8
2.2 Deep Reinforcement Learning . . . 9
2.2.1 Model-Based and Model-Free RLs . . . 10
2.2.2 Policy Gradient Methods Overview . . . 11
2.2.3 Proximal Policy Optimization Algorithm . . . 11
2.2.4 Deep Deterministic Policy Gradient Algorithm . . . 13

3 Financial Terms and Concepts **16**
3.1 Portfolio and Portfolio Optimization . . . 16
3.1.1 Asset . . . 16
3.1.2 Stocks . . . 17
3.1.3 Portfolio . . . 17
3.1.4 Portfolio Optimization . . . 17
3.2 Financial Time Series . . . 17
3.2.1 Price . . . 18
3.2.2 Returns . . . 19
3.3 Technical Indicators . . . 20
3.3.1 Simple Moving Average (SMA) . . . 20
3.3.2 Moving average convergence divergence (MACD) . . . 21
3.3.3 Relative Strength Index (RSI) . . . 21
3.3.4 Commodity Channel Index (CCI) . . . 22

3.3.5 Directional Movement Index (DMI) . . . 22
3.4 Performance Criteria . . . 23
3.4.1 Annual Return . . . 23
3.4.2 Cumulative Return . . . 23
3.4.3 Annual Volatility . . . 23
3.4.4 Shape Ratio . . . 24
3.4.5 Maximum Drawdown . . . 24

4 Experimental Setup 24
4.1 Data description . . . 25
4.1.1 Data Pre-Processing . . . 26
4.2 Methodology . . . 26
4.2.1 Assumptions . . . 26
4.2.2 Libraries . . . 27
4.3 Environment . . . 27
4.3.1 State . . . 27
4.3.2 Action . . . 28
4.3.3 Reward Function . . . 28
4.3.4 The DRL Models . . . 29

5 Results 31
5.1 Performance Comparison . . . 31
5.1.1 PPO Backtest Results . . . 31
5.1.2 DDPG Backtest Results . . . 33
5.1.3 PPO vs DDPG Performance . . . 36

6 Conclusions and Future Work 40
6.1 Conclusions . . . 40
6.2 Future Work . . . 40

Appendix 41

A Technical Indicators 41
A.1 Commodity Channel Index (CCI) . . . 41
A.2 Directional Movement Index (DMI) . . . 41

References 42

List of Figures

1 Reinforcement Learning control loop 4
2 MDP control loop 9
3 A deep reinforcement learning model relying on a fully connected multilayer feedforward neural network with one hidden layer (Mnih et al., 2016) 10
4 PPO algorithm (Schulman et al., 2017) 13
5 DDPG Architecture (Steven Li and Zeng, 2021) 15
6 DDPG Algorithm (Lillicrap et al., 2015) 16
7 PPO - Monthly returns % 31
8 PPO - Annual Returns 31
9 PPO - Distribution of monthly returns 31
10 PPO - Return Quantities 31
11 PPO Backtest Results 32
12 DDPG - Monthly returns % 33
13 DDPG - Annual Returns 33
14 DDPG - Distribution of monthly returns 33
15 DDPG - Return Quantities 34
16 DDPG Backtest Results 34
17 PPO weights/actions for each ETF in the portfolio 36
18 DDPG weights/actions for each ETF in the portfolio 36
19 PPO vs DDPG returns during the whole testing period 37
20 Percent Return by Portfolio 37

List of Tables

1 Values of hyperparameters - PPO 29
2 Values of hyperparameters - DDPG 30
3 Overall performance comparison 39

List of Abbreviations

Symbols

(+DI) Positive Directional Movement Line
(-DI) Negative Directional Movement Line

A

ADX Average Directional Movement Index

C

CAGR Compound Annual Growth Rate
CCI Commodity Channel Index

D

DDPG Deep Deterministic Policy Gradient Algorithm
DL Deep Learning
DMI Directional Movement Index
DNN Deep Neural Network
DPG Deterministic Policy Gradient
DQN Deep Q Network
DRL Deep Reinforcement Learning
DX Optional Directional Index

E

ETF Exchange-Traded Funds

M

MACD Moving average convergence divergence
MDP Markov Decision Process
ML Machine Learning
MSBE Mean-Squared Bellman Error

N

NN Neural Network

P

PPO Proximal Policy Optimization

R

RL Reinforcement Learning
RSI Relative Strength Index

S

SMA Simple Moving Average
?? ??

List of Notation

s	State
S	State space
o	Observation
a	Action
A	Action space
r	Return
μ	Deterministic policy
π	Stochastic policy
τ	Trajectory
ρ	State distribution
P	Transition function
R	Reward function
γ	Discount factor
$\mathbb{E}$	Expected return
π^*	Optimal policy
V	Value function
Q	Action-Value function
w	Weight
p	Price
$\overrightarrow{p}$	Price column vector
$\overrightarrow{P}$	Price matrix
$\overrightarrow{r}$	Return column vector
$\overrightarrow{R}$	Portfolio return column vector
$\overrightarrow{W}$	Portfolio weight matrix
R_p	Portfolio return value
R_f	Risk-free rate
σ_p	Portfolio standard deviation

Author overview

Section	Author
Abstract	Stavre
1. Introduction	Stavre
1.1. Problem Statement	Stavre
1.2. Motivation	Stavre
1.3. Report Structure	Stavre
2. Reinforcement Learning Background	Le
2.1. Reinforcement Learning: Key concepts and terminology	Le
2.2. Deep Reinforcement Learning	Stavre
2.2.4. Deep Deterministic Policy Gradient Algorithm	Maini
3. Financial Terms and Concepts	Le
4. Experimental Setup	
4.1. Data Description	Stavre
4.2. Methodology	Stavre
4.3. Environment	Stavre
4.3.1. State	Stavre
4.3.2. Action	Stavre
4.3.3. Reward Function	Stavre, Maini
4.3.4. The DRL Models	Stavre, Maini
5. Results	
5.1.1. PPO Backtest Results	Stavre
5.1.2. DDPG Backtest Results	Maini
5.1.3. PPO vs DDPG	Stavre, Maini
6. Conclusions and Future Work	Stavre

1 Introduction

Written by: Xhorxhia Stavre

Aside from traditional approaches like modern portfolio theory (Markowitz, 1968), portfolio managers frequently use engineering and sophisticated statistical methods to create more complex models. In recent years, the financial industry has become increasingly automated leading to a slew of benefits, such as more earnings and efficiency. This is a result of Artificial Intelligence (AI) and Machine Learning (ML) models becoming a new standard. Anyhow, the stock market is characterized by rapid changes and insufficient periodic data, so using a supervised learning model is not always the best solution. One of the most used and successful methods adapted in many research publication in resolving such issues is reinforcement learning (Li et al., 2020).

Reinforcement learning (RL) has been around for over 30 years and is the most comparable ML method to human-like learning. To put it differently, learning happens through trial and error. These types of methods involve an agent which is regularly supplied with new information from raw data and its goal is to maximize the value of a pre-determined reward function.

After the Deepmind's Alphago system defeated the world champion in the Go board game, it has grown in popularity as a method for generating financial market forecasts. RL initially appeared in the financial sector in 1997 and was quickly adopted (Moody et al., 1998). Many experimentations have been done using these techniques in order to learn about the optimum trade time through data in a number of scenarios, including stock and portfolio management. Moreover, another subfield called deep reinforcement learning (DRL), which combines Deep neural networks (DNN) and RL offers an advantage when it comes to portfolio management because the model avoids precisely forecasting future prices and optimizes portfolio performance using the provided reward function (Kang et al., 2018).

In this report, we are particularly interested in the portfolio management problem, which is the process of regularly reallocating a set of funds with the goal of maximizing the return while limiting risk (Reilly & Brown, 2011). We briefly define the report's goal and highlight the research and application fields from which we take inspiration in this introduction

chapter. As a result, this chapter sets the stage for the remainder of the report.

1.1 Problem Statement

Written by: Xhorxhia Stavre

The primary objective of this report is to investigate the effectiveness of two DRL agents in the portfolio management problem. Furthermore, we want to compare their performance and returns. Also, discover if the models will be able to detect market pattern and give the expected performance when compared to real life events.

We will create our portfolio with a limited number of stocks and a limited initial budget. Striving to maximize its total profit, our agents will learn how to allocate these funds in the most efficient way possible. The agents are trained and tested on real market data. The two DRL models are then compared with respect to their annual return, Sharpe ratio, annual volatility, maximum drawdown, and other metrics.

1.2 Motivation

Written by: Xhorxhia Stavre

Games, healthcare, and finance are just a few of the fields where RL can be used and many noteworthy implementations in these fields have been achieved. Also, some very important breakthroughs have been created by research groups such as DeepMind and OpenAI.

Inspired by these results, we wanted to try and train two different agents in order to maximize profits with little to no supervision. New states are offered to the portfolio management agent in order to interact with the stock market environment, taking actions. The environment provides feedback to the agent in the form of profit or loss for each action made.

The stock market is a great illustration of how we can assess our agent's ability to understand market dynamics over time and make the optimal trades to maximize its profit. Another reason why DRL agents might be ideal candidates for the portfolio management tasks is their capacity to focus on long-term objectives rather than short-term ones.

1.3 Report Structure

Written by: Xhorxhia Stavre

This section gives a brief outline of the report chapters:

Chapter 2: Reinforcement Learning

The objective of this chapter is to introduce the key concepts and terminologies behind the field of Reinforcement Learning, its application in finance and provide an explanation on the algorithms implemented on a later stage of the report.

Chapter 3: Financial Terms and Concepts

In this chapter, we present a brief overview of financial concepts that are required for a better understanding of the following contents.

Chapter 4: Experimental Setup

This chapter gives the details of how we formalize the portfolio management problem in the DRL framework and describe the dataset used. We define the main assumptions, types of models, and how the environment is built.

Chapter 5: Results

In this chapter we present and explain the results from different experiments.

Chapter 6: Conclusions and Future Work

This chapter provides a summary of the results and conclusions. The chapter concludes with a few comments concerning future study.

2 Reinforcement Learning Background

Written by: Thao Nguyen Le

Reinforcement learning (RL), supervised learning and unsupervised learning are the three basic paradigms of Machine learning (ML). RL is based on the concept of learning by interaction with surroundings. Unlike other types of ML, this area focuses on goal-directed learning from interaction.

The term ”computational method to acquire understanding and automating goal-oriented learning and decision making” is used to illustrate RL (Sutton & Barto, 2018). To optimize a reward signal, it learns to map states to actions. The RL agent is not taught how to behave in a given state; instead, it tries to figure out which actions are the most rewarding by trial-and-error. Trial-and-error search and delayed reward are the two most defining characteristics of reinforcement learning.

The two primary elements of a reinforcement learning system are the agent and environment. Besides, the system has four basic sub-elements: a policy, a reward signal, a value function, and, optionally, an environment model. At each time step, the agent (partially) observes the states of the environment before taking an action based on a policy that alters the environment's state. Afterwards, the environment sends a reward signal to the agent. Maximizing the total rewards is the objective of the agent. The policy is particularly important, when it comes to an RL system. It can be defined as a set of rules that instructs the agent on which action should be taken in a given state.

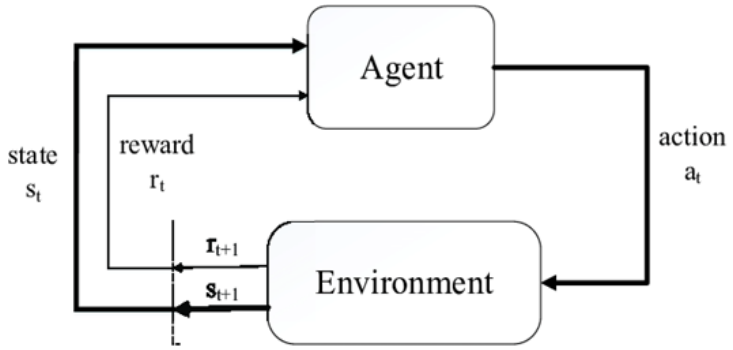

Figure 1: Reinforcement Learning control loop
(Graesser & Keng, 2019)

In Figure 1 we can see that (Graesser & Keng, 2019) the environment gives information about the *state* of the system. Then agent will interact with environment in order to observe the state and use the given information to take an *action*. The environment accepts the action and transitions to the next state. Then, it returns the next state and a *reward* to the agent. One *time step* is complete when the loop (*state* $\longrightarrow$ *action* $\longrightarrow$ *reward*) is finished. The loop repeats until the environment terminates.

Furthermore, the trade-off between exploitation and exploration, which does not exist in

supervised or unsupervised learning, is a basic challenge in RL. In order to receive rewards, the agent has to exploit what it has already learned, but it also has to explore to take better action in the next stage. The trial-and-error search feature is the source of this challenge. On the one hand, the agent should use past experiences to take the actions that are known to give high rewards, but also investigate which behaviors lead to better rewards.

Another property of reinforcement learning is that it is goal-oriented, which implies that while dealing with an uncertain environment, the learning agent explicitly examines the entire problem (Sutton & Barto, 2018). In that environment, such an agent may evaluate billions of scenarios, remembering which actions resulted in higher rewards.

Due to these advantages, it's perfect for dealing with complex financial markets because it requires minimum modeling and feature engineering.

2.1 Reinforcement Learning: Key concepts and terminology

Written by: Thao Nguyen Le

2.1.1 States and Observations

A state s is a comprehensive description of the environment's current state when there is no information about the environment that the state does not have access to. A partial description of a state is an observation o, which may omit information. The environment is considered as fully observed once the agent knows the entire state of the environment. Otherwise, it is called partially observed.

2.1.2 Action Space

The action space is the set of all valid actions in a given environment. Some environments contain discrete action spaces, in which the agent can only make a limited number of movements. Continuous action spaces exist in other environments when actions are real-valued vectors in continuous spaces, such as when the agent is controlling a robot in the real world.

2.1.3 Policies

The agent uses a policy, as a rule, to decide what action should be taken. There are two types of policies: deterministic denoted by μ and stochastic denoted by π which is given as the following:

$$a_t = \mu(s_t) \tag{1}$$

$$a_t \sim \pi(\cdot|s_t)0 \tag{2}$$

2.1.4 Trajectory/ Episodes

The sequence of states and actions is called trajectory τ, given as:

$$\tau = (s_0, a_0, s_1, a_1, \cdots) \tag{3}$$

with s_i, a_i being the state and action at time index i respectively.

The initial state of the environment s_0 is randomly sampled from the start-state distribution, being denoted by ρ as given:

$$s_0 \sim \rho_0(\cdot) \tag{4}$$

The transitions of states in the environment which results in the moving from state s_t at the time index t to state s_{t+1} at the time index $t+1$ are controlled by the environment's policy and depend on only the most recent action a_t. They are either deterministic or stochastic respectively as given:

$$s_{t+1} = f(s_t, a_t) \tag{5}$$

$$s_{t+1} \sim P(\cdot|s_t, a_t) \tag{6}$$

where, $P(\cdot|s_t, a_t)$ is the probability for the occurrence of the transition from s_t to s_{t+1}

2.1.5 Reward and Return

The reward function R is a key factor in a RL loop. It is built on current state s_t, the current action a_t, and the next state s_{t+1}, as given:

$$r_t = R(s_t, a_t, s_{t+1}) \tag{7}$$

The goal of the agent is to maximize the cumulative reward over trajectory τ. Let $R(\tau)$, be a discounted sum of the rewards in a trajectory τ over the time step to T, we get:

$$R(\tau) = r_0 + \gamma r_1 + \gamma^2 r_2 + \cdots + \gamma^T r_T = \sum_{t=0}^{T} \gamma^t r_t \qquad \gamma \in [0, 1] \tag{8}$$

The discount factor γ plays an important role that defines the value of future rewards. The larger γ, the more weight this rewards will receive in the future time steps (Graesser & Keng, 2019).

2.1.6 Value Functions

The information of the object is provided by the value function. This enables an agent to define the potential states and actions in term of estimating the expected return. They are formed by the following functions:

Value Function

$$V^{\pi}(s) = \mathbb{E}_{\tau \sim \pi}[R(\tau)|s_0 = s] \tag{9}$$

Equation (9) allows to define how good a state is. IT evaluates the expected return from the state s with the assumption that the agent continues to act based on its current policy π. The return $R(\tau)$ is calculated from the current state s to the end of an episode, because only rewards from state s are collected.

Action-Value Function

$$Q^{\pi}(s, a) = \mathbb{E}_{\tau \sim \pi}[R(\tau)|s_0 = s, a_0 = a] \tag{10}$$

Equation (10) allows to define how good a state-action pair is. It evaluates the expected

return by taking action a from the state s with the assumption that the agent continues to act based on its current policy π. The return is calculated from the current state s to the end of an episode because only rewards from state s are collected.

2.1.7 Markov Decision Process

When an agent transitions in the environment from a current state to the next one usage of a transition function is required. In RL, a transition function is generated as a Markov Decision Process (MDP) which is a mathematical framework (Graesser & Keng, 2019). MDP is the standard formalization of sequential decision making, in which actions influence not just present rewards but also future events. The Markov property (Silver, 2020) is satisfied by a state if and only if it provides a sufficient statistic for predicting the future, allowing the long-term history to be discarded.

MDP is an object containing S, A, P, R, γ, where $\boldsymbol{S}$ is a set of states (state-space), where each state s complies with the Markov property, $\boldsymbol{A}$ is action space of action a, $\boldsymbol{P}(s_{t+1}|s_t, a_t)$ is a state-transition function of the environment, $R(s_t, a_t, s_{t+1})$ is the reward function of the environment and γ is a discount factor.

Each stage of the MDP stores the current weight as well as the state of all assets. Actions are used to adjust asset weights from period to period, and rewards are defined by expressing how much expected return the portfolio generates in its current state.

RL has been defined as an MDP, so ML control loop can be presented as an MDP control loop as the below pseudo-code for algorithm :

```
1:  Given an env (environment) and an agent:
2:  for episode = 0, ..., MAX_EPISODE do
3:      state = env.reset()
4:      agent.reset()
5:      for t = 0, ..., T do
6:          action = agent.act(state)
7:          state, reward = env.step(action)
8:          agent.update(action, state, reward)
9:          if env.done() then
10:             break
11:         end if
12:     end for
13: end for
```

Figure 2: MDP control loop
(Graesser & Keng, 2019)

The code at Figure 2 shows how the agent and environment interact over episodes and time periods. The agent and the environment are reset (lines 3-4) at the start of every episode. Environment creates an initial state on $reset()$. Based on the given state agent then interacts with the environment to take an action (line 6). Next, a new state and reward (line 7) are generated by the environment at the new time step.

2.2 Deep Reinforcement Learning

Written by: Xhorxhia Stavre

Deep Reinforcement Learning (DRL) models are known to combine the decision-making capacity of RL with the perception of Deep Learning (DL). DL allows for complex feature engineering and end-to-end learning via gradient descent, reducing the need for domain expertise for specific tasks (Li et al., 2020). Deep neural networks are used to approximate RL functions such as value function, $\hat{v}(s, \theta)$, policy function, $\pi(a|(s, \theta))$, state transition function and reward function, where θ parameters are the weights of the deep neural network. (Li, 2017).

A DNN is a neural network architecture with more than one hidden layer and the neural

network should be trained with repeated samples of data in order for it to give results that are close to what we expect (LeCun et al., 2015). Inputs are feed to the neural network during the training phase and after the amount of divergence (error) in the results is calculated an output is created. An Objective Function or Loss Function is used in order to calculate the divergence and the goal of the neural network (NN) is to decrease the amount of inaccuracy in the output. The weights of the NN change themselves during the training phase so the output can be close to the training data. If the network performs well, it means the NN has learned to its full potential.

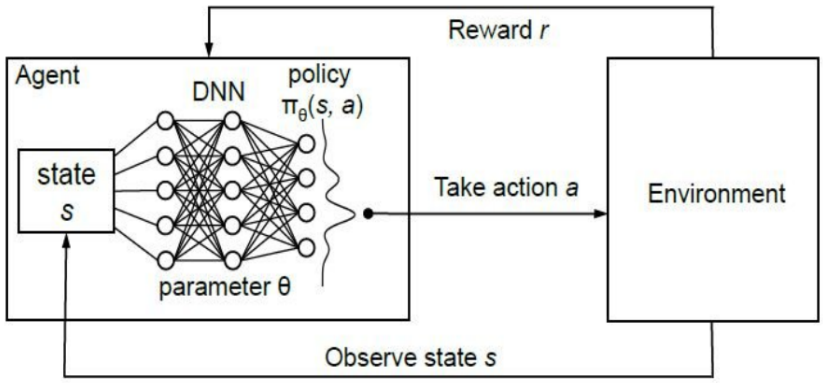

Figure 3: A deep reinforcement learning model relying on a fully connected multilayer feed-forward neural network with one hidden layer (Mnih et al., 2016)

2.2.1 Model-Based and Model-Free RLs

Written by: Xhorxhia Stavre

A key difference between model-free and model-based RL methods is the pattern of learning and planning (Huys et al., 2014). ***Model-based*** methods try to determine how the environment reacts to the chosen action and maximise the reward, while ***model-free*** methods are concerned with maximizing the reward inconsiderately of future states (Bach et al., 2018). In a trading scenario, we aim to draw conclusions about how the environment evolves from historical data without having the ability to influence it with actions and this will the be the approach used in this report.

2.2.2 Policy Gradient Methods Overview

Written by: Xhorxhia Stavre

Policy gradient methods compute an estimator for the policy gradient and afterward input it into a stochastic gradient ascent algorithm (Schulman et al., 2017).

Policy Gradient Algorithms (PG) can be Policy-based or Actor-Critic: (Ashwin Rao, 2020)

- ***Policy-based:*** In policy-based methods, the policy is learned by using a function approximation for the policy instead of learning a value function.
- ***Actor-Critic:*** In the other hand, in an actor-critic method, first a policy is learned using a function approximation for the policy called actor and then the value function is learnt using a function approximation for the value function called critic.

2.2.3 Proximal Policy Optimization Algorithm

Written by: Xhorxhia Stavre

The Proximal Policy Optimization (PPO) algorithm improves a policy as much as possible without sacrificing efficiency (Schulman et al., 2017). PPO is built on first-order methods in order to keep the new and old policy as near as possible. It is an improved and easier to implement version of TRPO (Trust Region Policy Optimization) (Schulman et al., 2015).
PPO is a ***policy optimization*** method, which means it directly aims to learn the best policy. This makes it less prone to failures and, as a result, reliability is increased.
Secondly, PPO is an ***on-policy*** method, which implies that the information obtained from each action is only used once to update the policy. By introducing a clipping function this policy is changed smoothly, preventing "bad" actions and quick updates from happening. In order to maintain the new policy as close to the old one, PPO performs a clipping operations in the objective function. In order to optimize the objective Gradient Descent methods can be applied.

Finally, PPO is also an ***actor-critic*** method, meaning that two neural networks are necessary, an actor and a critic-network.
The critic-network receives and evaluates the reward, whereas the actor-network is used in

the learning step. A loss function makes it possible to connect both networks. PPO uses a complex loss function with a clipping mechanism in order to prevent misfitting with the restriction of ϵ. ϵ is a hyperparameter that determines how far the new policy can change from the old one. Outside of the clipped span, PPO discourages major strategic shifts.
The policy weights θ are updated via:

$$\theta_{k+1} = \underset{\theta}{\operatorname{argmax}} \underset{s,a\sim\pi_{\theta k}}{\mathbb{E}} [L(s, a, \theta_k, \theta)], \tag{11}$$

L is defined as

$$[L(s, a, \theta_k, \theta)] = min(r_t(\theta)\hat{A}_t, clip(r_t(\theta)), 1 - \epsilon, 1 + \epsilon)\hat{A}_t \tag{12}$$

where

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{old}(a_t|s_t)}, \tag{13}$$

and

$$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_t + ... + ... + (\gamma\lambda)(T - 1 + 1)\delta_{T-1}, \tag{14}$$

and

$$\delta_t = r_t + \gamma V_{(s_{t+1})} - V_{(st)} \tag{15}$$

The clip function is used to keep the ratio $r_t(\theta)$ within the range [1-ϵ, 1+ϵ]. $r_t(\theta)$ is the probability ratio between the new and old policy (Karunakaran, 2020). The L function takes the minimum between the original ratio and its clipped version. As a result, by preventing huge updates from happening at each step, PPO improves the whole network preparation. λ is equal to 1, V stands for the value function, and $\hat{A}_t$ is the advantage function, so if $\hat{A}_t$ is larger than 1, the current action is better than another action at this state. The formulas can be further investigated at (Schulman et al., 2017) and (Mnih et al., 2016).
The pseudo-code of for algorithm is shown in Figure 4.

Algorithm 1 PPO, Actor-Critic Style

```
for iteration=1, 2, ... do
    for actor=1, 2, ..., N do
        Run policy π_θold in environment for T timesteps
        Compute advantage estimates Â_1, ..., Â_T
    end for
    Optimize surrogate L wrt θ, with K epochs and minibatch size M ≤ NT
    θ_old ← θ
end for
```

Figure 4: PPO algorithm (Schulman et al., 2017)

[1]

2.2.4 Deep Deterministic Policy Gradient Algorithm

Written by: Deepanshu Maini

Deep Deterministic Policy Gradient (DDPG) (Lillicrap et al., 2015) is a reinforcement learning algorithm that combines the abilities of DQN (Deep Q Network) and DPG (Deterministic Policy Gradient). It combines the DQN's Q-learning which is the ability to calculate Q-function and DPG's ability to get a deterministic policy to iterate over continuous action space.

DDPG is an off-policy model-free actor-critic algorithm. It's an off-policy algorithm as it uses two different policies for exploration and updates of the policy. It includes a stochastic behavioural policy for the exploration and deterministic policy for the target update. It's a model-free algorithm as it does not use transition probability distribution and reward function associated with MDP. In DDPG the optimal policy is chosen on the basis of trial and error experienced by the agent. It's an actor-critic algorithm as it has two networks which are actor and critic network. The actor network decides which action should be taken and critic network evaluate the action and informs how good or bad was the action to the actor network.

As it comes to the Q learning side of DDPG the Q function needs to be evaluated. The Q function provides an expected discounted reward that we get when an action a is executed

[1] We will not discuss TRPO in this report. A more detailed discussion can be found in (Schulman et al., 2015)

in state s and a policy is followed. This function is also known as Bellman equation. Using this Q function optimal deterministic action can be found.

$$a^*(s) = arg\,max_a Q^*(s,a) \tag{16}$$

Bellman Equation:

$$Q^*(s_t, a_t) = E[r + \gamma max_{a_{t+1}} Q^*(s_{t+1}, a(t+1))] \tag{17}$$

Where a is an action in the state s for the reward r and γ is the discount factor and $\gamma \in [0, 1]$. The discount factor influences the convergence.
Now to know how close our approximator/prediction maker which is a neural network, came to satisfying the Bellman equation we use a loss function. It is also known as Mean-Squared Bellman Error (MSBE). Q-learning in DDPG is performed by minimising the MSBE loss with stochastic gradient descent. We write this function as:

$$L = E[(Q_\phi(s_t, a_t) - (r + \gamma max_{a_{t+1}} Q_\phi(s_{t+1}, a_{t+1})))^2] \tag{18}$$

In the case of Discrete action spaces, Q-values for each action can be calculated separately and can be compared easily for the optimal max value which makes Q-learning ideal for discrete action spaces. When it comes to Continuous action spaces, evaluation and individual comparison for each Q-value is very demanding and computationally expensive and this leads to unstable learning. Which is why DDPG deploys a target policy network that compute an action that nearly maximises the target. The goal for DDPG is to minimise the MSBE while maximising the target value, which is expressed as:

$$r + \gamma max_{a_{t+1}} Q_\phi(s_{t+1}, a_{t+1}) \tag{19}$$

In DDPG, the target network is updated once per main network update by polyak averaging:

$$\phi_{targ} \leftarrow \tau\phi_{targ} + (1 - \tau)\phi \tag{20}$$

where τ is a hyperparameter between 0 and 1 (usually close to 1). This hyperparameter is called polyak. The equations can be investigated further at (Lillicrap et al., 2015) and Up, 2018.

The architecture of DDPG contains the DDPG agent and the replay buffer Steven Li and Zeng, 2021. The DDPG agent consists of the actor network and the critic network. The actor network is responsible for making the action. It is updated using gradient ascent to find the action policy that maximises the return. The critic network is responsible for estimating the expected rewards on the basis of current state and actions. It is updated using gradient descent on MSBE with the target value. The actor network is responsible for making the action. It is updated using gradient ascent to find the action policy that maximises the return. The Critic Network is responsible for estimating the expected rewards on the basis of current state and actions. It is updated using gradient descent on MSBE with the target value. The actor-target network and the critic-target network are synchronised through the soft updates. Moreover, When an action is taken the transition (s, a, s', r) observation is stored in the replay buffer, where s is state, a is action, s' is next state and r is reward. These transitions are randomly batched for sampling. The scheme of the architecture can be seen in Figure 5:

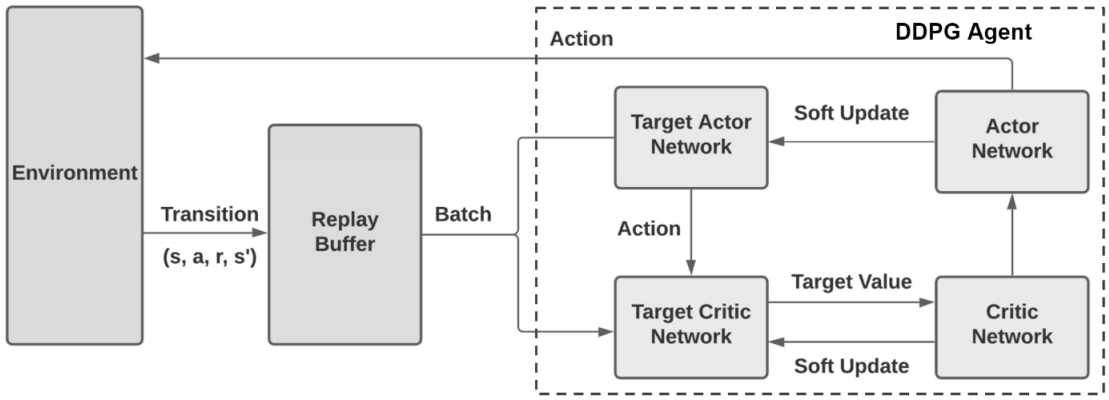

Figure 5: DDPG Architecture (Steven Li and Zeng, 2021)

The pseudo-code of for algorithm is shown in Figure 6 :

Algorithm 1 DDPG algorithm

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ.
Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q$, $\theta^{\mu'} \leftarrow \theta^\mu$
Initialize replay buffer R
for episode = 1, M **do**
  Initialize a random process $\mathcal{N}$ for action exploration
  Receive initial observation state s_1
  for t = 1, T **do**
    Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise
    Execute action a_t and observe reward r_t and observe new state s_{t+1}
    Store transition (s_t, a_t, r_t, s_{t+1}) in R
    Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R
    Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$
    Update critic by minimizing the loss: $L = \frac{1}{N}\sum_i (y_i - Q(s_i, a_i|\theta^Q)^2)$
    Update the actor policy using the sampled gradient:

$$\nabla_{\theta^\mu}\mu|_{s_i} \approx \frac{1}{N}\sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu}\mu(s|\theta^\mu)|_{s_i}$$

    Update the target networks:

$$\theta^{Q'} \leftarrow \tau\theta^Q + (1-\tau)\theta^{Q'}$$
$$\theta^{\mu'} \leftarrow \tau\theta^\mu + (1-\tau)\theta^{\mu'}$$

  end for
end for

Figure 6: DDPG Algorithm (Lillicrap et al., 2015)

3 Financial Terms and Concepts

Written by: Thao Nguyen Le

3.1 Portfolio and Portfolio Optimization

3.1.1 Asset

An asset is a resource that has economic value that an individual or organization holds or controls with the expectation that it will create profit in the long run. Typical examples of assets are stocks, accrued incomes, cash, etc. Within the scope of this research project, we focus on stocks but still comply with general principles that apply to all kinds of assets.

3.1.2 Stocks

A stock also known as an "equity" or "share", is a type of security that reflects proportionate ownership in the issuing company. The stockholder is entitled to that percentage of the corporation's assets and earnings. Stocks are the foundation of practically every portfolio and are traded mostly on stock exchanges.

3.1.3 Portfolio

A portfolio is a set of financial assets such as stocks, bonds, commodities, currencies, and cash equivalents, etc. Non-publicly tradable securities, such as real estate, art, and private investments, can also be included in a portfolio. Investors hold portfolios directly, while financial advisors and money managers manage them. Investors should build their investment portfolios based on their risk tolerance and investment objectives. A portfolio is a collection of several financial assets that can be expressed in a vector format as follows:

- **Constituents:** M assets of which it concludes
- **Portfolio vector $w_{i,t}$:** its i-th element reflects the ratio of the total budget invested to the i-th asset, such that:

$$w_t = [w_{1,t}, w_{2,t}, \ldots, w_{M,t}]^T \in R^M \text{ and } \sum_{i=1}^{M} w_{i,t} = 1 \tag{21}$$

3.1.4 Portfolio Optimization

Portfolio optimization is the process of finding the optimum portfolio (also known as asset distribution) from a set of all possible portfolios based on some objective. The objective is to maximize elements like expected return while minimizing costs like financial risk. Our study defines the portfolio optimization problem by finding the values for the weights $w_{t,i}$ such that portfolio value is maximized overtime under certain constraints.

3.2 Financial Time Series

Prices change throughout time due to the dynamic character of the economy and non-static supply and demand balance. This enables market dynamics to be considered as time series

and to be analyzed and modeled using technical methods and tools.

3.2.1 Price

Let the price of an asset at the time index t is $p_t \in \mathbb{R}$, the sequence of a univariate time-series is $p_1, p_2, \cdots, p_T$ and the price of an i-th asset at the time index t is $p_{i,t}$. Let T is the length of the time of asset i, so its price time series is the column vector $\overrightarrow{p}_{i,1:T}$ represented as:

$$\overrightarrow{p}_{i,1:T} = \begin{bmatrix} p_{i,1} \\ p_{i,2} \\ \vdots \\ p_{i,T} \end{bmatrix} \in \mathbb{R}_+^T \tag{22}$$

For a portfolio, the price vector p_t is defined as:

$$\overrightarrow{p}_t = \begin{bmatrix} p_{1,t} & p_{2,t} & \cdots & p_{M,t} \end{bmatrix} \in \mathbb{R}_+^T \tag{23}$$

Applying the single asset time series notation to the multivariate one enables to create the asset price matrix $\overrightarrow{P}_{1:T}$ by stacking column-wise the T-samples price time-series of the portfolio's M assets, we get:

$$\overrightarrow{P}_{1:T} = \begin{bmatrix} \overrightarrow{p}_{1,1:T} & \overrightarrow{p}_{2,1:T} & \cdots & \overrightarrow{p}_{M,1:T} \end{bmatrix} = \begin{bmatrix} p_{1,1} & p_{2,1} & \cdots & p_{M,1} \\ p_{1,2} & p_{2,2} & \cdots & p_{M,2} \\ \vdots & \vdots & \ddots & \vdots \\ p_{1,T} & p_{2,T} & \cdots & p_{M,T} \end{bmatrix} \in \mathbb{R}_+^{T \times M} \tag{24}$$

Prices can be divided in 4 subtypes:

1. *High price* is the security's highest traded price during the day.
2. *Low price* is the security's lowest traded price during the day.
3. *Opening price* is the price of the security at the start of the trading day.
4. *Closing price* is the price of the security at the end of the trading day.

3.2.2 Returns

Investors do not benefit directly from absolute asset prices. More specifically, the change of the p prices over time is significantly important because they illustrate efficiently the profit and loss of an investment, or, to put it differently, its return.

Gross return

The scaling factor of an investment in an asset at the time $t-1$ is represented by the gross return R_t. Consider this example: at time $t-1$ the investment in an asset is X dollars, then this investment will worth $R_t X$ dollars at time t. Hence:

$$R_t = \frac{p_t}{p_{t-1}} \in \mathbb{R} \tag{25}$$

where, p_{t-1} and p_t are the price of the asset at time $t-1$ and t respectively.

Simple return

The simple return, r_t, is a more generally used term that indicates the percentage change in asset price from time $t-1$ to time t, as follows:

$$r_t = \frac{p_t - p_{t-1}}{p_{t-1}} = R_t - 1 \in \mathbb{R} \tag{26}$$

The column vector below represents the T-samples simple returns time-series of the i-th asset.

$$\overrightarrow{r}_{i,1:T} = \begin{bmatrix} r_{i,1} \\ r_{i,2} \\ \vdots \\ r_{i,T} \end{bmatrix} \in \mathbb{R}^T \tag{27}$$

While the simple returns vector r_t is represented as the following:

$$\overrightarrow{r}_t = \begin{bmatrix} r_{1,t} \\ r_{2,t} \\ \vdots \\ r_{M,t} \end{bmatrix} \in \mathbb{R}^M \tag{28}$$

Where $r_{i,t}$ is the simple return of the $i-$th asset at time index t.Therefore, we get the following equation at time index t:

$$r_t = \sum_{i=1}^{M} w_{i,t} r_{i,t} = w_t^T r_t \in \mathbb{R} \tag{29}$$

We generate the simple return matrix $\overrightarrow{R}_{1:T}$ by stacking column-wise the T-samples simple returns time-series of the M assets of the portfolio, using the price matrix in (24) and the definition of simple return in (26):

$$\overrightarrow{R}_{1:T} = \begin{bmatrix} \overrightarrow{r}_{1,1:T} & \overrightarrow{r}_{2,1:T} & \cdots & \overrightarrow{r}_{M,1:T} \end{bmatrix} = \begin{bmatrix} r_{1,1} & r_{2,1} & \cdots & r_{M,1} \\ r_{1,2} & r_{2,2} & \cdots & r_{M,2} \\ \vdots & \vdots & \ddots & \vdots \\ r_{1,T} & r_{2,T} & \cdots & r_{M,T} \end{bmatrix} \in \mathbb{R}^{T \times M} \tag{30}$$

After taking the column vector $\overrightarrow{R}_{1:T}$ of the portfolio matrix for the time range $t \in [1, T]$ we multiply $\overrightarrow{R}_{1:T}$ with a portfolio weights matrix $\overrightarrow{W}_{1:T}$ then extract the T diagonal elements of the product we will obtain the portfolio return time-series as the following:

$$r_{1:T} = \text{diag}(\overrightarrow{R}_{1:T} \overrightarrow{W}_{1:T}) \in \mathbb{R}^T \tag{31}$$

3.3 Technical Indicators

3.3.1 Simple Moving Average (SMA)

SMA is the average of stock closing price over a past period (Price, 2021). SMA makes it easy to determine trend direction by smoothing price data. Trading signals are triggered by the crossing of the price line and SMA. When the SMA crosses above the price line the

decision of going short or exit-long should be considered and when the SMA crosses below the price line a decision of going long or covering-short should be made.

3.3.2 Moving average convergence divergence (MACD)

MACD is a trading indicator used in technical analysis of stock prices and was created by Gerald Appel in the late 1970s (Appel, 2005). Its goal is to reveal changes in the strength, direction, momentum, and duration of a stock's price trend.

This indicator is generated from historical price data, commonly by the closing price of stocks and contains three time-series elements:

1. MACD Line: commonly generated by 12-period EMA subtracts 26-period EMA
2. Signal Line: commonly 9-period EMA of MACD Line
3. MACD Histogram: generated by MACD Line subtracts Signal Line

where, EMA (exponential moving average) (Chen, 2022) is a weighted moving average that measures the trend of a financial security over a certain period of time. The security may be sold when the MACD line crosses below the signal line and bought when the MACD crosses above the signal line.

3.3.3 Relative Strength Index (RSI)

RSI (Wilder, 1978) evaluates the magnitude of recent price fluctuations to determine if a stock or other asset is overbought or oversold. This indicator has its value in the range of 0 to 100. The calculation is as follows:

$$\begin{aligned} \text{RSI} &= 100 - \frac{100}{1 + \text{RS}} \\ \text{where, RS} &= \frac{\text{Average Gain}}{\text{Average Loss}} \end{aligned} \tag{32}$$

Traditional interpretation and usage of the RSI are that it is considered overbought when above 70 and oversold when below 30. However, this traditional setting-up can also be modified to better fit the security if needed.

3.3.4 Commodity Channel Index (CCI)

CCI is a momentum-based oscillator aimed for seasonal commodities (Schlossberg, 2006). It measures the gap between the present price and the historical average price. CCI can fluctuate between -100 and 100 in most cases. When the indicator goes from the negative to the positive range, a bullish signal is generated, and when it falls from the positive to the negative range, a bearish signal is generated. The CCI indicator's default setting is 14, which means it will compare recent price changes to average price changes over 14 time periods. It can be calculated as the following:

$$\text{CCI} = \frac{\text{Typical Price} - \text{MA}}{0.015 \times \text{Mean Deviation}} \tag{33}$$

A more in depth explanation can be found in the Appendix A.1.

3.3.5 Directional Movement Index (DMI)

DMI shows how both directions of an asset's price and strength fluctuate and this aims to reduce false signals. The indicator compares prior highs and lows by creating two lines: a positive directional movement line (+DI) and a negative directional movement line (-DI) based on previous highs and lows. The average directional indicator ADX (Wilder, 1978) is an optional third line that can be used to measure the strength of an uptrend or downtrend by smoothing average of DX. When (+DI) exceeds (-DI), the price is under greater upward pressure than downward pressure. If the (-DI) is higher than the (+DI), the price is under more downward pressure. This indication might help traders figure out which way the trend is going. Crossovers between the lines are used occasionally as buy or sell signals.
A more in depth explanation can be found in the Appendix A.2.

3.4 Performance Criteria

3.4.1 Annual Return

For each portfolio the annual return is calculated as bellow:

$$\text{CAGR} = \left(\left(\frac{\text{EV}}{\text{BV}} \right)^{\frac{1}{t}} \right) - 1 \tag{34}$$

where CAGR is the compound annual growth rate, EV stands for ending value, BV stands for beginning value and t is the number of time periods calculated by year.

CAGR is defined as the year-over-year constant growth rate over a specified period of time (Pandey & Desai, 2016). It is one of the most reliable methods for calculating returns for investment portfolios.

3.4.2 Cumulative Return

The cumulative return of an investment is the total amount gained or lost by the daily return, regardless of the length of time involved. The cumulative return is expressed as a percentage, as per the following formula:

$$CR = \frac{\text{Final value} - \text{Initial value}}{\text{Initial value}} \tag{35}$$

Where final value and initial value is respectively the final and the initial value of the portfolio.

3.4.3 Annual Volatility

The annual volatility (standard deviation) is the annualized volatility which refers to the variance in the value of an asset over the period of a year. This measurement reflects the degree of risk associated with a certain investment. Because there are 252 trading days in one year, the annual volatility of the portfolio is defined by multiplying the daily volatility by $\sqrt{252}$ (Menezes, 2017).

3.4.4 Shape Ratio

The Sharpe Ratio (Sharpe, 1994) is the average return over the risk-free rate per unit of volatility or total risk. It is measured by deducting the risk-free rate of the portfolio's return and then dividing the result by the portfolio's excess return standard deviation.
The Sharpe Ratio can be calculated as per following:

$$\text{Sharpe Ratio} = \frac{R_p - R_f}{\sigma_p} \tag{36}$$

where R_p is the portfolio return, R_f is the risk-free rate and σ_p is the standard deviation of the portfolio.

3.4.5 Maximum Drawdown

Maximum Drawdown is the maximum loss observed by the greatest movement from a high point to the low point of a portfolio value before a new peak is reached and measures the downside risk over a certain period of time (Hayes, 2020). It is presented in percentages term as the following formula:

$$MDD(T) = max\{0, max_{t\in(0,t)}P(t) - P(T)\} \tag{37}$$

where, $MDD(T)$ is the maximum drawdown at the end of the period and $P(T), P(t)$ are the prices of stock at time T and t.

4 Experimental Setup

The sections that follows describes the approach to optimize a stock portfolio using two model-free reinforcement learning methods mentioned in Section 2.2.3 and Section 2.2.4.
First, a description of the data sources and pre-processing will be presented, followed by methodologies, assumptions and a detailed explanation of the environment and more.

4.1 Data description

Written by: Xhorxhia Stavre

For the present report the data was extracted from Yahoo Finance! APIs to compile a dataset composed of daily high, low, open, closing prices, as well as volume traded. The U.S. market (S&P 500), Emerging Markets and Eurozone ETFs (Exchange-Traded Funds) were selected to form the dataset used. The tickers used are "IVV" for S&P 500, "EZU" for Eurozone and "EEM" for Emerging Markets.

An ETF is a collection of assets that can be traded on a daily basis in the stock market. Some advantages of investing in ETFs are diversification, tax efficiency, transparency, and minimal management costs (Deville, 2008). ETFs follow a certain index, sector, or commodity and they may be bought and sold on a stock market just like any other stock.

The information in the dataset is available for each of the ETFs in each trading day for the total period of 11 years (2010-2021). A desctiption of each ETF used can be found below:

1. ***iShares Core** S&P **500 ETF*** tracks the results of an index composed of large-capitalization U.S. equities (iShares by BlackRock, 2021a).

2. ***iShares MSCI Emerging Markets ETF*** tracks the investment results of an index composed of large-and-mid-capitalization emerging markets equities (iShares by BlackRock, 2021b).

3. ***iShares MSCI Euzone ETF*** tracks the investment results of an index composed of large-and-mid-capitalization equities from developed market countries in Europe (iShares by BlackRock, 2021c)

We divide the data as:

- ***Total Period:*** 01-01-2010 to 01-01-2021
- ***Train Period:*** 01-01-2010 to 01-01-2018
- ***Test Period:*** 01-01-2018 to 01-01-2021

4.1.1 Data Pre-Processing

Written by: Xhorxhia Stavre

Data preprocessing is an important step in developing a high-quality machine learning model. In order to turn the data into a model-ready condition, we must check for missing data and do feature engineering.
Various information, such as historical stock prices,dates, current holding shares, technical indicators, and more, must be considered in a real trading scenario.
Finally, the data is normalized by calculating the natural logarithm of each element to improve training stability.

4.2 Methodology

Written by: Xhorxhia Stavre

A Markov decision process is used to describe the trading scenario. The stock market acts as the environment and the DRL models act as the agents. The agent takes daily actions in the environment, such as selecting a stock from the portfolio and determining its weight. These actions are based on the agent's observations. A more in depth explanation about the models and the environment can be found in the sections below.

4.2.1 Assumptions

Written by: Xhorxhia Stavre

To assess an asset the portfolio management problem, we must backtest it, so we estimate its performance as if it had been applied to the past data, without taking into account future data.. This necessitates a set of assumptions about historical conditions, which we will outline below. These assumptions are very important in the real world, but we have made some simplifications.

- ***No Market Impact***

  The herd effect is a phenomenon caused when large orders have an impact in the stock market behaviour. We assume that the orders made by the agents have no market impact, so no significantly large orders are made.

- ***No Transaction Costs***

  Transaction costs are present in the real world and important for the trading process, however we don't use them in our model creation.

  We decided to discard them and assume they do not have an impact in the profits. The agent is expected to modify the portfolio weights or allocations without additional costs.

- ***No Margin Trading***

  When the portfolio is constructed the initial value (1_000_000) is specified and no other input is considered.

4.2.2 Libraries

Written by: Xhorxhia Stavre

The open-source library from FinRL is used to create our trading environment and test our algorithms (Liu et al., 2020). In addition to FinRL, we use the implementations of Stables Baselines3, which provides the algorithms used, DDPG and PPO (Raffin et al., 2019).

4.3 Environment

Written by: Xhorxhia Stavre

The environment has to be built up first in order for the models to be trained.
The next stage is to create a training environment, which includes a data representation suited for NN training and a reward function to define performance for each time step. The environment keeps track of the daily returns and the portfolio value at each time step. In this part, we'll go through these prerequisites in detail.

4.3.1 State

Written by: Xhorxhia Stavre

The covariance matrix is used to decide which stocks to include in the portfolio and give the starting wights for each of them. It describes the standard deviation of a portfolio and it's the model's first state. The covariance is calculated based on the price movements in a year,

but since this cannot be computed for the first year the data in 2010, this year is dropped. If the covariance between two stocks is positive, they will move in the same direction, but if the covariance is negative, they will move in opposite ways. Additional approaches can also be considered for identifying the best performing stocks, but one of the most important ideas of RL is to have minimal prior information, in order for the agent to learn by interacting with the environment.
Next, we added the technical indicators mentioned in Section 3.3 to give more states for the agent to move.

4.3.2 Action

Written by: Xhorxhia Stavre

The action a indicates the weight of each stock in the portfolio. So, the agent observes the state of the market at each time step and decides how to redistribute portfolio weights based on this knowledge. Since the weights should be equal to 1,a is within [0,1]. The action, a_t is represented with the weight vector w_t, leading to $\sum_{i=0}^{n} a_{i,t} = 1$. We assume a long-only portfolio with non-negative weights.

4.3.3 Reward Function

Written by: Xhorxhia Stavre, Deepanshu Maini

The reward function r(s, a, s') provides the agent with a foundation for improving its actions. To compute the portfolio's reward we use the entire amount owned by the agent and the daily returns. The environment obtains an action at each time step, which is the portfolio weights w_t, which computes the returns as:

$$r_t = \vec{w}_{t-1} \cdot R_t \tag{38}$$

where R_t is the daily return at time t or rate of return. Now, if we assume the environment trades at the end of the day: $R_t = \frac{p_t^c}{p_{t-1}^c} - 1$, where p^c can be explained as the closing price. To summarize, the rewards at time t are given by the current return R_t and the portfolio weights in the previous time step, w_{t-1}. When this is done for the whole portfolio, all the

tickers/stocks provided are included and the portfolio return at a given time t is calculated as:

$$PR_t = \sum_{i=1}^{n} \vec{w}_{t-1i} \cdot R_{ti} \tag{39}$$

where PR_t is the portfolio return and n is the number of stocks provided for the portfolio from the ticker list. When this portfolio return is multiplied with the current portfolio value, we get the profit/loss made over the whole portfolio for the actions.

4.3.4 The DRL Models

Written by: Xhorxhia Stavre, Deepanshu Maini

We primarily train two separate agents based on the reinforcement learning framework. As stated previously we employed PPO and DDPG algorithms.

Hyperparameters

Learning a DRL model in the financial field usually involves exploring a stochastic environment, therefore creating a replicable model is difficult. Same trials were repeated to assess the model's best performing parameters. Parameters with strong empirical findings were chosen. The hyperparameters that were employed are listed in the table below:

Table 1: Values of hyperparameters - PPO

`n_steps`	20480
`total_timesteps`	2_000_000
`learning_rate`	0.001
`ent_coef`	0.01
`gamma`	0.99
`vf_coef`	0.5
`batch_size`	5120
`gae_lambda`	0.95

,where batch_size is the minibatch size for learning the policy, ent_coef is the entropy coefficient, vf_coef is the value function coefficient for the loss calculation, learning_rate represents how fast we want the policy to be be learned. total_timesteps are the steps taken in

the environment, for this reason they need to be very high while training the PPO model since its main goal is to explore the environment and to find the best policy as it goes. Most of the parameters were previously explained in section 2.2.3, so more information can be found there.

Table 2: Values of hyperparameters - DDPG

`learning_rate`	0.0005
`total_timesteps`	500000
`batch_size`	128
`buffer_size`	50000

here learning_rate represents how fast we want the policy to be be learned, we don't want learning_rate to be high as we don't want get stuck on a local maxima/minima while the policy is learning, total_timesteps is the total number of samples to train on, buffer_size represents maximum number of elements that will be buffered when prefetching and these elements will be randomly selected, batch_size is the size of the batch for learning the policy.

5 Results

5.1 Performance Comparison

In this section, the performance comparison results are shown. This research included performing back-testing for the individual agents (PPO and DDPG). As mentioned in 4.2.1, backtesting is used for determining the feasibility of a trading strategy by examining how it would perform using previous data.

We have used the library pyfolio to create summary tear sheets and thematic plots that combine key portfolio metrics (Quantopian-Inc., n.d.).

5.1.1 PPO Backtest Results

Written by: Xhorxhia Stavre

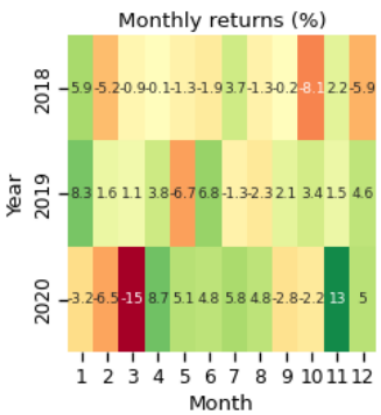

Figure 7: PPO - Monthly returns %

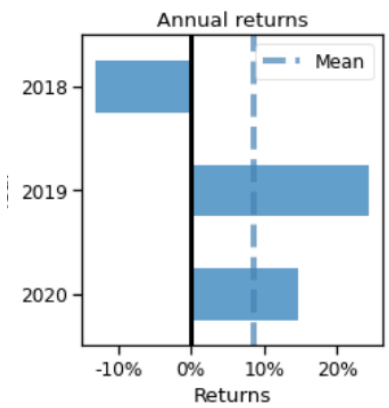

Figure 8: PPO - Annual Returns

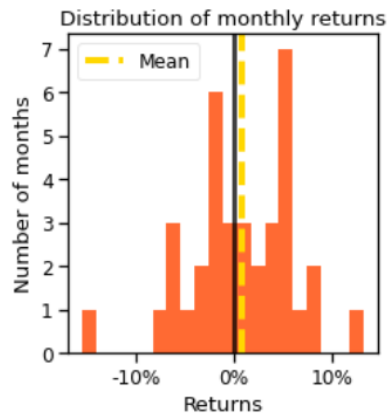

Figure 9: PPO - Distribution of monthly returns

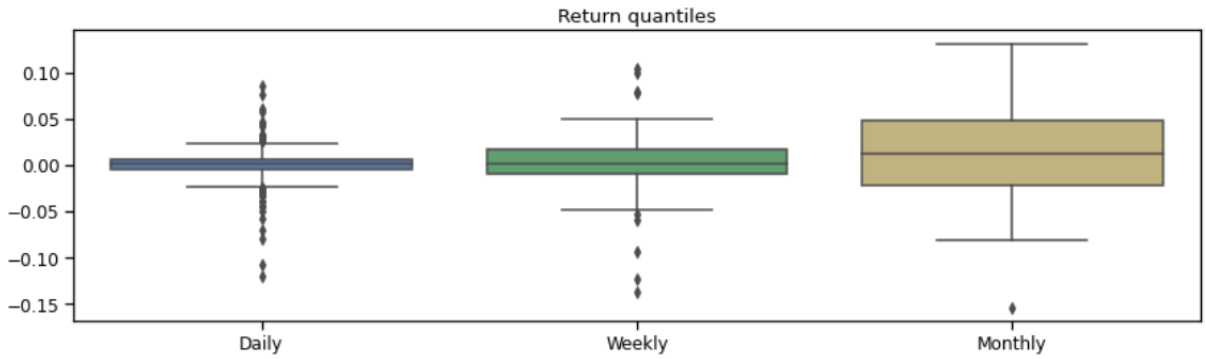

Figure 10: PPO - Return Quantities

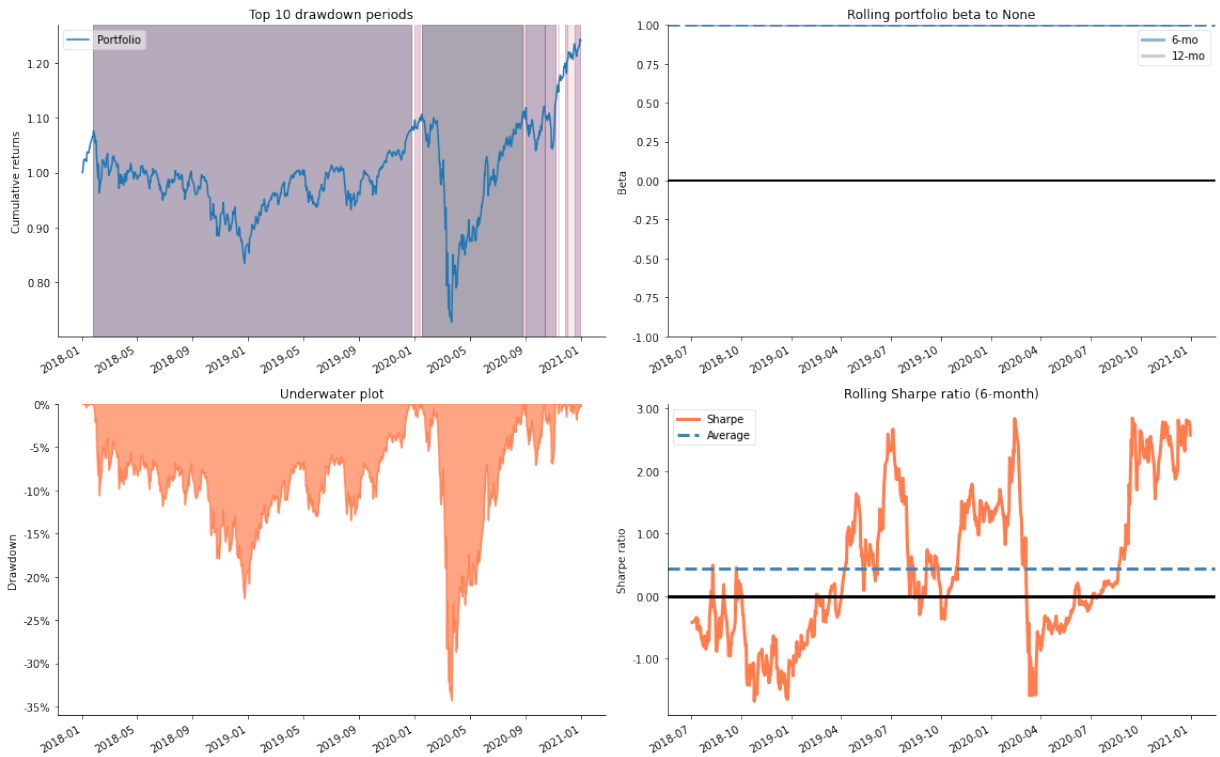

Figure 11: PPO Backtest Results

We have more monthly returns than losses, as shown in Figure 7, although two uncommon events are detected in March 2020 and November 2020. This conclusion is supported by Figure 9, which shows the two events in the distribution's tails. The lowest value (-15) was caused by the Covid-19 crisis, while the highest value (+13) was caused by the new market regulations taking place in the following period of 2020. Furthermore, we may conclude that the overall returns are positive because they are greater than the mean. However, having such noticeable extreme values leads to kurtosis, which means the probability in the tails in higher than expected.

Figure 8, on the other hand, shows a positive yearly return in the years 2019 and 2020, but a very poor return in the year 2018.

Figure 10 shows the returns in relation to outliers or extreme events. Many outliers may be found in daily returns, but not all of them can be considered as extreme events, and the same can be said for the weekly returns. However, the crisis can clearly be seen in the monthly return boxplot.

Maximum drawdown explains the difference from the highest value of the portfolio to the

lowest. In Figure 11 we can see the top 10 drawdown periods with the lowest period being March 2020 and the highest one November 2020.

The Rolling Sharpe ratio shows how this ratio is moving through time. It can be seen that in 2018 there is a very low Sharpe ratio, meanwhile from April 2019 we have a positive one. The highest value is a Sharpe ratio of 3 which can be translated as 1% risk = 3% returns and this profit is very high. In the Underwater plot we can see a high drawdown in January 2019 and a bigger one in March 2020.

Finally, we did include the graph for the Rolling portfolio beta for illustration reasons but it is worth mentioning that in real life beta can never be lower than 0, since our portfolio consists only of ETFs and they are a measurement of the whole market. So, beta can have a better explanation when individual stocks are used in the portfolio.

5.1.2 DDPG Backtest Results

Written by: Deepanshu Maini

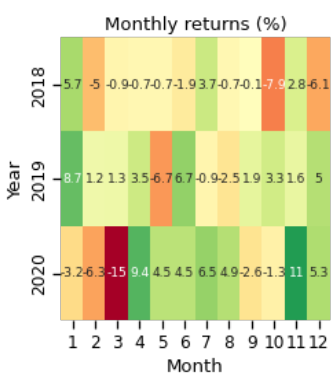

Figure 12: DDPG - Monthly returns %

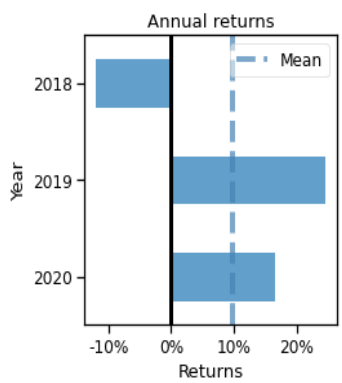

Figure 13: DDPG - Annual Returns

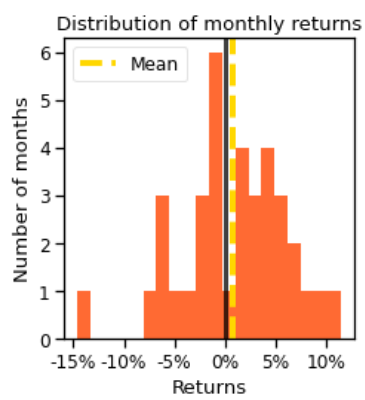

Figure 14: DDPG - Distribution of monthly returns

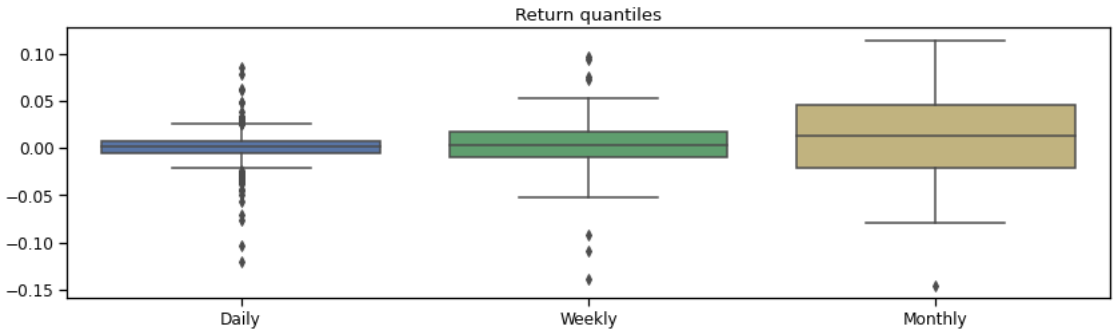

Figure 15: DDPG - Return Quantities

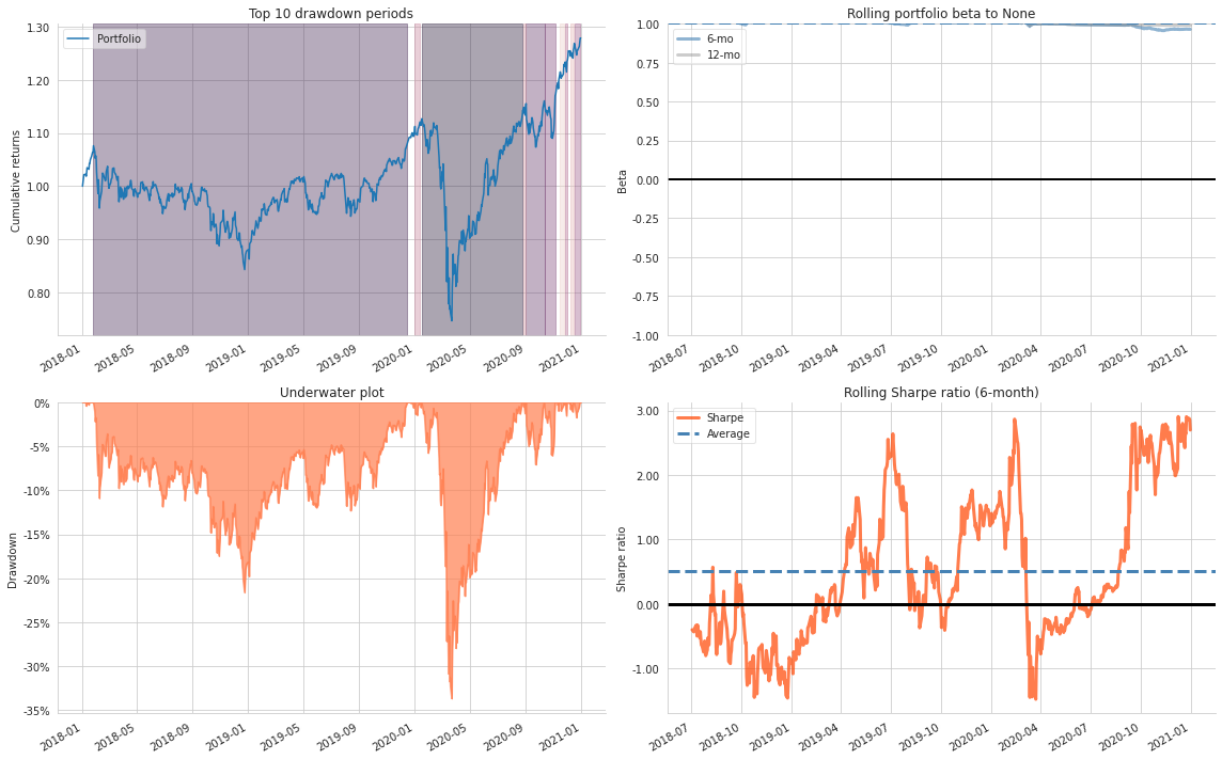

Figure 16: DDPG Backtest Results

The Backtest results for the DDPG model are gained by putting the trained model through the test period of 3 years from 01 Jan 2018 to 01 Jan 2021.

The drastic impact of the Covid-19 on the stock markets can be clearly observed in the month of March 2020 in Figure 12. We can also see the recovery after March 2020 and how the markets adapted. The months of March, April and November 2020 are very rare as the monthly returns are (-15), (+9.4) and (+11) respectively, and such high numbers represent

huge shifts in the world economy. This could also be seen in Figure 14 which are at the extreme ends of the graph, also a positive mean is observed which means the agent has more returns than losses.

Figure 13 shows that we got negative annual return in 2018, and positive annual return in 2019 and 2020.

Beta is a measurement of volatility of an asset in comparison to the movement of the whole market. Which means if beta is greater than 1, then the asset value is more volatile than the market it is, if beta is equal to 0, then the asset value is very correlated to the market and if the beta is lower than 1, then the asset value is less volatile than the market. As we see in Figure 16, the Rolling portfolio beta, the value of beta in the rolling window of 6 and 12 months of the testing phase has stayed very close to 1. This is due to the assets we used for this study, since ETFs are markets themselves. Underwater plot in Figure 16 represents the asset's with values lesser than their notional value over the testing phase. In Figure 16, all three graphs, the Underwater Plot, Rolling Sharpe ratio graph and Drawdown periods graph shows the impact of Covid-19 from the month of March 2020 and the economic recovery after that, which can be correlated with the same observable impact in Figure 12 which is mentioned before as well.

5.1.3 PPO vs DDPG Performance

Written by: Xhorxhia Stavre, Deepanshu Maini

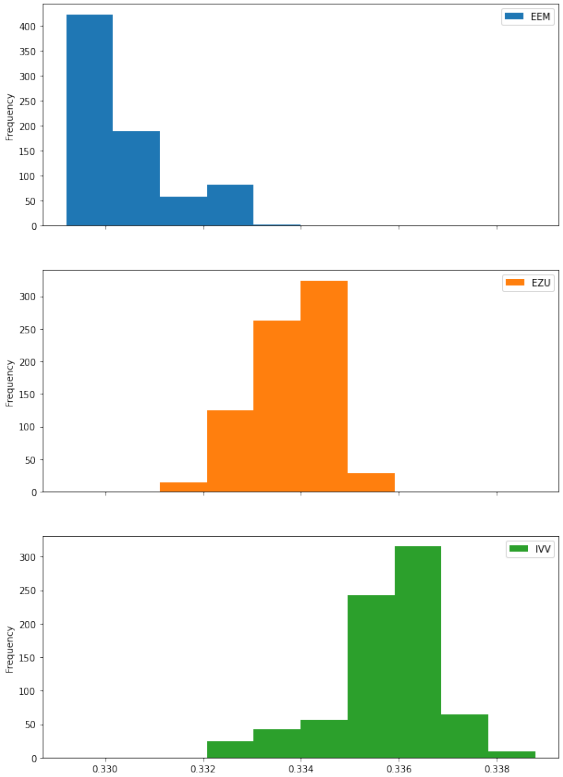

Figure 17: PPO weights/actions for each ETF in the portfolio

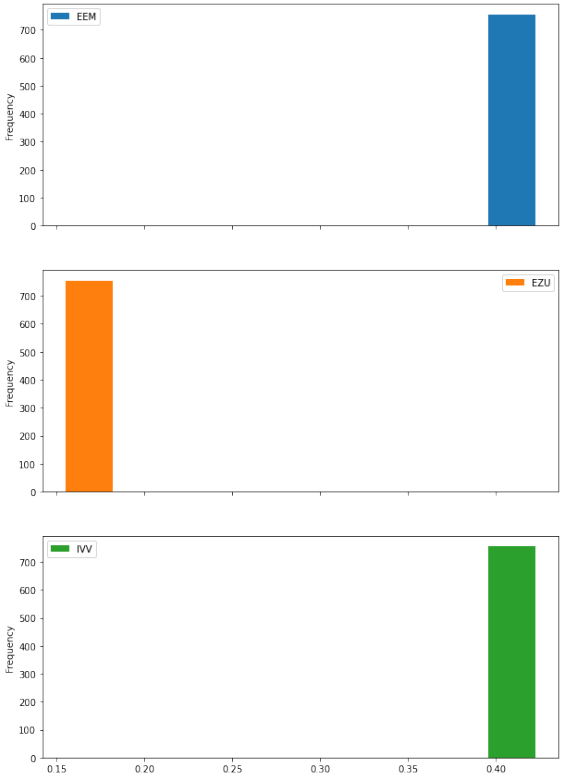

Figure 18: DDPG weights/actions for each ETF in the portfolio

where, ”EEM” stands for Emerging Markets ETF, ”EZU” stands for Eurozone ETF and ”IVV” stabds for S&P 500 ETF.

Figure 19: PPO vs DDPG returns during the whole testing period

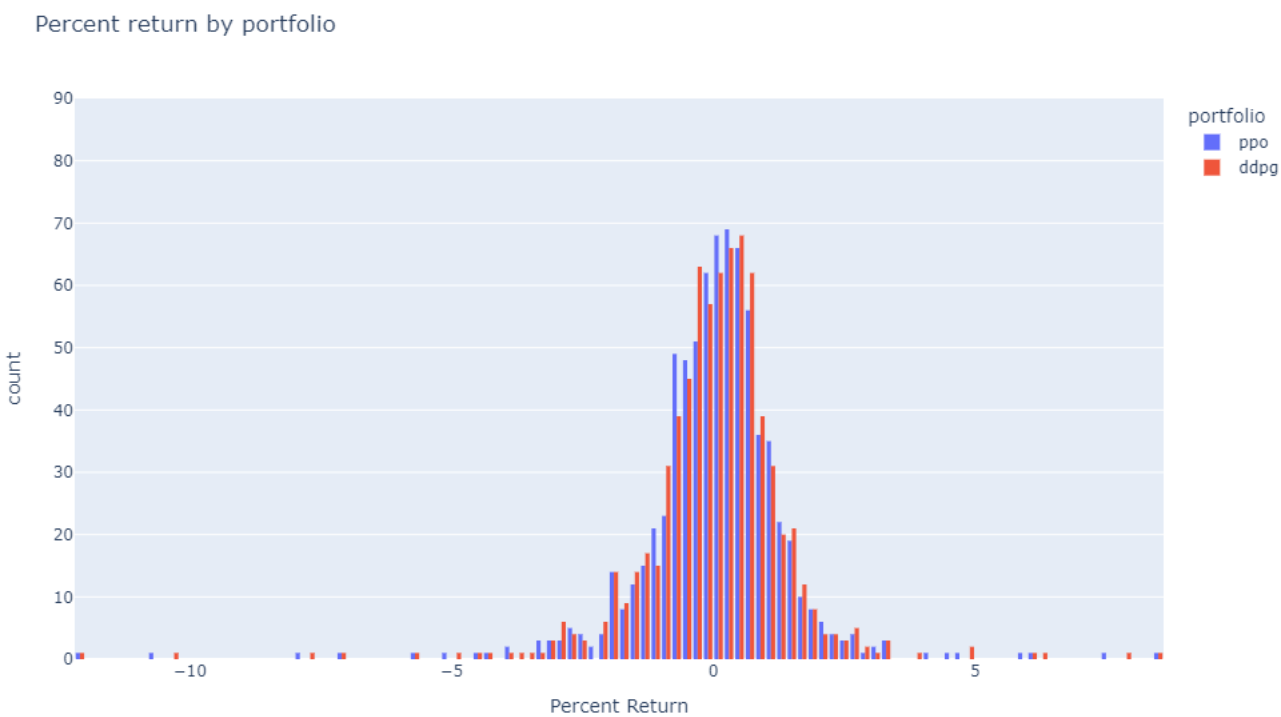

Figure 20: Percent Return by Portfolio

PPO and DDPG seem to show very similar patterns when it comes to daily returns which can be seen in Figure 19, however slight differences in the performance metrics are noticed. The Sharpe ratio of 0.43, means 1% risk = 0.43% returns. The Sharpe ratio is low, since our portfolios consist of ETFs, which take into account aggregate markets and this makes their return lower than using individual stocks. The annual return illustrates how much the portfolio has increased or decreased over the course of a year. During the testing period, the DDPG portfolio had the greatest average annual returns (8.5%) and PPO has a lower ones (7.5%). The annual volatility assesses how risky the portfolios are, as well as how volatile their returns are. The largest annual volatility was in the DDPG portfolio (22.7%), whereas PPO showed a slightly lower annual volatility (22.7%). The Sharpe ratio is used to calculate risk-adjusted returns. As a result, a greater Sharpe ratio over a lower Sharpe ratio is preferable, so we can state that DDPG has higher returns than PPO.

However, when it comes to the weights(actions, in our environment) normalized over the portfolio that can be seen in Figure 17 and 18, we can see a very different outcome in the algorithms. Since, PPO tries to improve the policy during the whole exploration process, the weights tend to differ a lot in certain periods depending on which ETF would provide the higher return. So, the more returns a certain stock can provide the higher will be the amount invested there.

In the case of DDPG algorithm, the Q function seems to have been able to help the deterministic policy function find the optimal policy very quickly and doesn't seem to update after that. This suggest that optimal policy seems to have been found and a better policy can not be learned from the given data set.

Although, the massive difference between actions of PPO and DDPG does not seem to have effects on the daily returns produced by these algorithms, which can be clearly seen in Figure 19 through Cumulative returns.

In Figure **??** we can see many outliers in the data and as stated before these are the result of the rare/extreme event happening in the market. So not all the data can be described by the curve in the normal distribution. The performance of both algorithms in a three year period, from 2018-2021 is summarized in the table below:

Performance Metric	**PPO**	**DDPG**
Annual return(%)	7.483237	8.530083
Cumulative return(%)	24.171580	27.835184
Annual volatility(%)	22.772399	22.842660
Sharpe ratio	0.432249	0.473892
Calmar ratio	0.218299	0.253343
Maximum drawdown(%)	-34.279778	-33.670083

Table 3: Overall performance comparison

To conclude, the findings suggest that the deployed approaches are effectively learning to follow profitable market patterns, and the performance metrics support this. Also, if we compare the models performance to real life events we can state that the prediction is very accurate.

In the testing period we see low returns in 2018 and these came as a result of the bear market declines. President Donald Trump's trade war with China, the global economic slowdown, and the Federal Reserve raising interest rates too soon, were just a few of the factors leading to this (Frazee, 2018).

In the other hand, we notice the market crash in March 2020 from the Covid-19 crisis and in November 2019 a big market gain. The conclusion of the presidential race, as well as promising vaccine developments helped for this. The Federal Reserve lifted its policy rate by a quarter of a percentage point. It was the central bank's first significant action to contain inflation and particularly, technology stocks, benefited from the move (Davis, 2022). Moreover, when we look at the weight of each ETF in Figure 17 and Figure 18, it is noticeable that the percentage invested in the U.S. market (S&P500) is higher during the end of the testing period so in the year 2020. This will lead to higher returns since it is the largest market and the most profitable one.

6 Conclusions and Future Work

6.1 Conclusions

Written by: Xhorxhia Stavre

The focus of this report was to investigate either a DRL approach could be utilized to improve the performance of a stock portfolio. Two DRL algorithms, PPO and DDPG, were employed to generate two portfolios in order to accomplish this. S&P 500, Eurozone, and Emerging Markets ETFs were used to build the portfolios and the strategies were evaluated using a variety of performance metrics. When compared to each other, DDPG surpasses PPO in all tests. DDPG achieves higher cumulative returns (+27.8%) and risk-return ratios (0.43 Sharpe ratio, 0.21 Calmar ratio). Also, when compared to real life events the result generated are very similar, which shows that the agents where able to learn and make predictions as accurately as possible.

Despite showing a good performance, these approaches have big limitations when used in the real world. More information should be added in the models in order for them to be successful and profitable in a real-world situation, such as transaction fees or data sources other than stock prices. Despite these limitations, the results of this study show that machine learning methodologies may be used in portfolio management.

6.2 Future Work

Written by: Xhorxhia Stavre

Apart from the ones already mentioned, there are a few appealing elements that might lead to a more in depth investigation on the subject. Future research might explore the relevance of DRL techniques in portfolio management using a bigger set of data containing longer testing period, a larger index, or the inclusion of other financial aspects, such as financial news, as inputs to train the algorithms. Moreover, a variety of reward function can be used in order to determine the one that produces the best results.

Appendix

A Technical Indicators

A.1 Commodity Channel Index (CCI)

Typical Price $= \sum_{i=1}^{P} \frac{\text{High + Low + Close}}{3}$, P: Number of periods

MA: Moving Average

Moving Average $= \frac{\sum_{i=1}^{P} \text{Typical Price}}{P}$

Mean Deviation $= \frac{\sum_{i=1}^{P} \mid \text{Typical Price} - \text{MA} \mid}{P}$

A.2 Directional Movement Index (DMI)

$+\text{DI} = \frac{\text{Smoothed } +\text{DI}}{ATR} \times 100$

$\text{-DI} = \frac{\text{Smoothed -DI}}{\times} 100$

$\text{DX} = \frac{|+\text{DI} - \text{-DI}|}{+\text{DI} + \text{-DI}} \times 100$

$\text{Smoothed } +/\text{-DM} = \sum_{t=1}^{14} \text{DM} - \frac{\sum_{t=1}^{14} \text{DM}}{14} + \text{CMD}$

+DI (Directional Movement) = Current High − Previous high

-DI = Previous Low − Current Low

CMD: Current Directional Movement

ATR: Average True Range

References

Appel, G. (2005). *Technical analysis: Power tools for active investors*. Financial Times/Prentice Hall.

Ashwin Rao, T. J. (2020). *Foundations of reinforcement learning with applications in finance*. Stanford University.

Bach, F., Sutton, R., & Barto, A. (2018). Reinforcement learning: An introduction (adaptive computation and machine learning series).

Chen, J. (2022). *Exponential moving average (ema) as of 03/2022*. https://www.investopedia.com/terms/e/ema.asp

Davis, W. P. (2022). *Stocks notch their best week since november 2020*. https://www.nytimes.com/2022/03/18/business/economy/stock-market-today.html

Deville, L. (2008). Exchange traded funds: History, trading, and research. *Handbook of financial engineering* (pp. 67–98). Springer.

Frazee, G. (2018). *6 factors that fueled the stock market dive in 2018*. https://www.pbs.org/newshour/economy/making-sense/6-factors-that-fueled-the-stock-market-dive-in-2018

Graesser, L., & Keng, W. (2019). *Foundations of deep reinforcement learning: Theory and practice in python*. Pearson Education. https://books.google.de/books?id=0HW7DwAAQBAJ

Hayes, A. (2020). *Maximum drawdown (mdd) as of 2020*. https://www.investopedia.com/terms/m/maximum-drawdown-mdd.asp

Huys, Q. J., Cruickshank, A., & Seriès, P. (2014). Reward-based learning, model-based and model-free. *Encyclopedia of computational neuroscience* (pp. 1–10). Springer New York.

iShares by BlackRock. (2021a). *Ishares core s&p 500 etf, fact sheet as of 31/12/2021*. https://www.ishares.com/us/literature/fact-sheet/ivv-ishares-core-s-p-500-etf-fund-fact-sheet-en-us.pdf

iShares by BlackRock. (2021b). *Ishares msci emerging markets etf, fact sheet as of 31/12/2021*. https://www.ishares.com/us/literature/fact-sheet/eem-ishares-msci-emerging-markets-etf-fund-fact-sheet-en-us.pdf

iShares by BlackRock. (2021c). *Ishares msci euzone etf, fact sheet as of 31/12/2021*. https://www.ishares.com/us/literature/fact-sheet/ezu-ishares-msci-eurozone-etf-fund-fact-sheet-en-us.pdf

Kang, Q., Zhou, H., & Kang, Y. (2018). An asynchronous advantage actor-critic reinforcement learning method for stock selection and portfolio management. *Proceedings of the 2nd International Conference on Big Data Research*, 141–145.

Karunakaran, D. (2020). *Proximal policy optimization(ppo)- a policy-based reinforcement learning algorithm*. https://medium.com/intro-to-artificial-intelligence/proximal-policy-optimization-ppo-a-policy-based-reinforcement-learning-algorithm-3cf126a7562d

LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *nature*, *521*(7553), 436–444.

Li, Y., Ni, P., & Chang, V. (2020). Application of deep reinforcement learning in stock trading strategies and stock forecasting. *Computing*, *102*(6), 1305–1322.

Li, Y. (2017). Deep reinforcement learning: An overview. *arXiv preprint arXiv:1701.07274*.

Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., & Wierstra, D. (2015). Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*.

Liu, X.-Y., Yang, H., Chen, Q., Zhang, R., Yang, L., Xiao, B., & Wang, C. D. (2020). Finrl: A deep reinforcement learning library for automated stock trading in quantitative finance. *arXiv preprint arXiv:2011.09607*.

Markowitz, H. M. (1968). *Portfolio selection*. Yale university press.

Menezes, R. (2017). *How to calculate the annualized volatility as of 19/04/2017*. https://pocketsense.com/calculate-annualized-volatility-1393.html

Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D., & Kavukcuoglu, K. (2016). Asynchronous methods for deep reinforcement learning. *International conference on machine learning*, 1928–1937.

Moody, J., Wu, L., Liao, Y., & Saffell, M. (1998). Performance functions and reinforcement learning for trading systems and portfolios. *Journal of Forecasting*, *17*(5-6), 441–470.

Pandey, A., & Desai, P. N. (2016). Emerging structure of stem cell research in india: An analysis of publication output, 1990-2014. *J. Sci. Res.*, *5*(1), 49–61.

Price, M. (2021). *What is a simple moving average? as of 10/2021*. https://www.thebalance.com/what-is-a-simple-moving-average-5207024

Quantopian-Inc. (n.d.). Github - pyfolio [Accessed 27-03-2022]. https://github.com/quantopian/pyfolio

Raffin, A., Hill, A., Ernestus, M., Gleave, A., Kanervisto, A., & Dormann, N. (2019). Stable baselines3.

Reilly, F. K., & Brown, K. C. (2011). *Investment analysis and portfolio management (text only)*. Cengage Learning.

Schlossberg, B. (2006). *Technical analysis of the currency market : Classic techniques for profiting from market swings and trader sentiment / boris schlossberg*. John Wiley & Sons Hoboken, N.J.

Schulman, J., Levine, S., Abbeel, P., Jordan, M., & Moritz, P. (2015). Trust region policy optimization. *International conference on machine learning*, 1889–1897.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.

Sharpe, W. F. (1994). The sharpe ratio, the journal of portfolio management. *Stanfold University, Fall*.

Silver, D. (2020). *Markov decision processes*. https://www.davidsilver.uk/wp-content/uploads/2020/03/MDP.pdf

Steven Li, X.-Y. L., & Zeng, Y. (2021). Elegantrl demo: Stock trading using ddpg [Accessed 14-03-2022]. https://medium.com/mlearning-ai/elegantrl-demo-stock-trading-using-ddpg-part-i-e77d7dc9d208

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction*. The MIT Press.

Up, O. S. (2018). Deep deterministic policy gradient [Accessed 30-03-2022]. https://spinningup.openai.com/en/latest/algorithms/ddpg.html

Wilder, J. W. (1978). *New concepts in technical trading systems*. Trend Research.